Naija Prime School

Sprint 1 — Identity & User Management

Implementation walk-through in Visual Studio Code

Author: Benjamin Fadina

Stack: .NET 10, Blazor Web App (Auto interactivity), Entity Framework Core 10, SQL Server, ASP.NET Core Identity, Radzen Blazor 10

Editor: Visual Studio Code with the C# Dev Kit

Repository: https://github.com/benjaminsqlserver/NaijaPrimeSchool

Contents

Contents....2

1. Overview....5

1.1 What Sprint 1 delivers....5

1.2 Architecture at a glance....5

2. Prerequisites and VS Code setup....5

2.1 Tooling....5

2.2 VS Code extensions....6

2.3 Verifying the toolchain....6

3. Scaffolding the solution....6

3.1 Creating the root folder and solution file....6

3.2 Creating the four projects....7

3.3 Project references....7

3.4 NuGet packages....7

3.5 Restore and first build....8

4. The Domain layer....8

4.1 Auditing & soft-delete contracts....9

4.2 Identity entities....9

4.3 Lookup tables — no enums....10

4.4 Role names — constants, not an enum....10

5. The Application layer....11

5.1 Cross-cutting abstractions....11

5.2 Data transfer objects....12

5.3 Service contracts....12

6. The Infrastructure layer....13

6.1 ApplicationDbContext....13

6.2 Model configuration....13
6.3 Auditing and soft delete inside SaveChanges....14
6.4 Dependency injection — AddInfrastructure....15
6.5 UserService....15
6.6 LookupService....16
6.7 DatabaseInitializer — migrate and seed....16
6.8 Creating the first migration....16
7. The Web host — wiring Blazor + Identity....17
7.1 Program.cs walk-through....17
7.2 CurrentUserAccessor — bridging HttpContext to DI....18
7.3 Logout endpoint....18
7.4 Revalidating authentication state....19
8. The Razor UI....19
8.1 App and Routes....19
8.2 _Imports.razor....20
8.3 MainLayout and NavMenu....20
8.4 Sign-in page — Login.razor....20
8.5 The Users list — Users.razor....21
8.6 Add user — CreateUser.razor....21
8.7 Edit user — EditUser.razor....21
8.8 Assign roles — UserRoles.razor....21
8.9 Roles catalogue — RolesPage.razor....21
8.10 Reset password — ResetPasswordDialog.razor....21
8.11 Home dashboard....22
9. Theming — the Nigerian green + gold skin....22
10. Running the application locally....22

10.1 appsettings.json....22

10.2 Restore, build, run....23

10.3 A suggested VS Code launch configuration....23

11. Source control and pushing to GitHub....23

11.1 Writing a sensible .gitignore....23

11.2 First commit on the sprint branch....23

11.3 Pushing to the empty GitHub repo....24

12. Appendix — project map....24

1. Overview

This document is the narrative I wrote while building Sprint 1 of the Naija Prime School management system end-to-end in Visual Studio Code. It walks through every meaningful decision — how I scaffolded the solution, how the four Clean Architecture layers fit together, how ASP.NET Core Identity is wired into Blazor, how the Radzen UI is assembled, and how the whole thing finally lands on GitHub. A developer following the chapters in order should be able to recreate the codebase from an empty folder.

1.1 What Sprint 1 delivers

- Cookie-based authentication backed by ASP.NET Core Identity, with Guid keys.
- Seven seeded application roles: SuperAdmin, HeadTeacher, Teacher, SchoolBursar, SchoolStoreKeeper, Parent, Student.
- Role-based authorization policies and a role-scoped sidebar.
- Lockout on repeated failed sign-ins (5 attempts per 15 minutes).
- Revalidating authentication state — deactivated users are signed out within the revalidation window without needing to restart the app.
- A SuperAdmin-only user management workspace: list, search, filter, create, edit, activate / deactivate, reassign roles, and reset password.
- Auditing (CreatedOn/By, ModifiedOn/By, DeletedOn/By) applied automatically inside SaveChanges.
- Soft delete with EF Core global query filters so deleted rows are invisible to every ordinary query.
- Lookups (titles, genders, roles) as first-class tables — no enums.
- A custom Nigerian-themed green + gold Radzen skin.

1.2 Architecture at a glance

The solution follows a classic Clean Architecture layering. Each layer references only the layers strictly inward of it, so the Domain layer is pure POCOs with no knowledge of ASP.NET, EF Core, or the UI.

```
src/
├── NaijaPrimeSchool.Domain/          # Entities, base types, interfaces
├── NaijaPrimeSchool.Application/     # DTOs, service contracts, abstractions
├── NaijaPrimeSchool.Infrastructure/  # EF Core, Identity stores, service impls
├── NaijaPrimeSchool.Web/             # Blazor server host, auth, pages
└── NaijaPrimeSchool.Web.Client/      # Blazor WebAssembly client (Auto mode)
```

Dependency direction flows inward: Web → Infrastructure → Application → Domain. The Web.Client project is a peer of Web; the Web project references it so that Blazor Auto interactivity can share components between server and WASM.

2. Prerequisites and VS Code setup

Before I opened VS Code for the first line of code I made sure the tools below were on my machine. Everything listed is free.

2.1 Tooling

- .NET 10 SDK (10.0.201 or newer) — the project targets net10.0.

- SQL Server — any of SQL Server LocalDB, Developer / Express / Standard, or Azure SQL. I used LocalDB on my laptop and a local SQL Server instance on my development box.
- Git for Windows (brings the credential manager used when pushing to GitHub).
- GitHub CLI (gh) — optional, but handy for creating the remote and running pull-request commands from the terminal.
- Visual Studio Code.

2.2 VS Code extensions

In the Extensions side-bar I installed these five, in this order:

1. C# Dev Kit (Microsoft) — brings the C# language server, a solution explorer, and test explorer. This is the one essential extension.
2. C# (ms-dotnettools.csharp) — installed automatically with C# Dev Kit.
3. .NET Install Tool — manages .NET runtimes the Dev Kit needs.
4. IntelliCode for C# Dev Kit — AI-assisted completions.
5. EditorConfig for VS Code — honours the .editorconfig (if one exists).

I did not install any Blazor-specific extension: VS Code recognises .razor files through C# Dev Kit, and Radzen ships with its own IntelliSense via the NuGet package.

2.3 Verifying the toolchain

In a VS Code integrated terminal I confirmed everything:

```
dotnet --version         # 10.0.201 or higher
dotnet --list-sdks       # net10.0 listed
git --version
gh --version             # optional
sqlcmd -S .\SQLEXPRESS -Q "SELECT @@VERSION"   # or (localdb)\MSSQLLocalDB
```

3. Scaffolding the solution

VS Code does not have the project wizards Visual Studio does, so every project was created from the integrated terminal using the .NET CLI. The advantage is that the commands are reproducible and scriptable.

3.1 Creating the root folder and solution file

```
mkdir NaijaPrimeSchool
cd NaijaPrimeSchool
code .                                # open this folder in VS Code
dotnet new sln --format slnx -n NaijaPrimeSchool
```

I used the modern XML-flavoured .slnx format rather than the legacy .sln; both load in Visual Studio and in the C# Dev Kit solution explorer.

3.2 Creating the four projects

Each project lives under src/. I used the classlib template for pure libraries and the Blazor Web App template for the front door.

```
mkdir src
cd src

dotnet new classlib -n NaijaPrimeSchool.Domain         -f net10.0
dotnet new classlib -n NaijaPrimeSchool.Application    -f net10.0
dotnet new classlib -n NaijaPrimeSchool.Infrastructure -f net10.0

# Blazor Web App with Auto interactivity — produces both Web and Web.Client
dotnet new blazor -n NaijaPrimeSchool.Web \
    --interactivity Auto \
    --auth Individual \
    --use-local-db false \
    -f net10.0

cd ..
dotnet sln NaijaPrimeSchool.slnx add \
    src/NaijaPrimeSchool.Domain \
    src/NaijaPrimeSchool.Application \
    src/NaijaPrimeSchool.Infrastructure \
    src/NaijaPrimeSchool.Web \
    src/NaijaPrimeSchool.Web.Client
```

The blazor template generated both NaijaPrimeSchool.Web (the server host) and NaijaPrimeSchool.Web.Client (the WebAssembly counterpart) side-by-side. I chose --auth Individual so the template scaffolded a ready-to-customise Identity UI, and I set --use-local-db false because I prefer to wire the connection string explicitly.

3.3 Project references

The dependency graph is strictly inward:

```
cd src
dotnet add NaijaPrimeSchool.Application    reference NaijaPrimeSchool.Domain
dotnet add NaijaPrimeSchool.Infrastructure reference NaijaPrimeSchool.Application
dotnet add NaijaPrimeSchool.Web            reference NaijaPrimeSchool.Infrastructure
dotnet add NaijaPrimeSchool.Web            reference NaijaPrimeSchool.Web.Client
cd ..
```

3.4 NuGet packages

These are the package references I added to each project. Versions follow the net10.0 release line.

Domain

```
dotnet add src/NaijaPrimeSchool.Domain package Microsoft.Extensions.Identity.Stores --
version 10.0.0
```

Infrastructure

```
dotnet add src/NaijaPrimeSchool.Infrastructure package Microsoft.EntityFrameworkCore
--version 10.0.0
dotnet add src/NaijaPrimeSchool.Infrastructure package
Microsoft.EntityFrameworkCore.SqlServer  --version 10.0.0
dotnet add src/NaijaPrimeSchool.Infrastructure package Microsoft.EntityFrameworkCore.Tools
--version 10.0.0
dotnet add src/NaijaPrimeSchool.Infrastructure package
Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 10.0.0
```

I also added <FrameworkReference Include="Microsoft.AspNetCore.App" /> to the Infrastructure csproj so that ASP.NET Identity abstractions are available without pulling the full web SDK.

Web

```
dotnet add src/NaijaPrimeSchool.Web package Radzen.Blazor
--version 10.3.1
dotnet add src/NaijaPrimeSchool.Web package
Microsoft.AspNetCore.Components.WebAssembly.Server
dotnet add src/NaijaPrimeSchool.Web package
Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore
dotnet add src/NaijaPrimeSchool.Web package
Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

Web.Client

```
dotnet add src/NaijaPrimeSchool.Web.Client package Radzen.Blazor
--version 10.3.1
dotnet add src/NaijaPrimeSchool.Web.Client package
Microsoft.AspNetCore.Components.WebAssembly.Authentication
```

3.5 Restore and first build

```
dotnet restore
dotnet build
```

Everything compiled, leaving me with an empty but structurally correct Clean Architecture solution ready to grow.

4. The Domain layer

Domain contains the company's language: entities, value types, and the interfaces that describe cross-cutting concerns like auditing and soft delete. It has a single package reference — Microsoft.Extensions.Identity.Stores — which gives me access to IdentityUser<TKey> and IdentityRole<TKey> base classes without pulling in EF Core.

4.1 Auditing & soft-delete contracts

I defined two tiny interfaces under Common/ so any entity can opt in independently:

```
// Common/IAuditable.cs
public interface IAuditable
{
    DateTimeOffset CreatedOn { get; set; }
    string? CreatedBy { get; set; }
    DateTimeOffset? ModifiedOn { get; set; }
    string? ModifiedBy { get; set; }
}
```

```
// Common/ISoftDelete.cs
public interface ISoftDelete
{
    bool IsDeleted { get; set; }
    DateTimeOffset? DeletedOn { get; set; }
    string? DeletedBy { get; set; }
}
```

Then BaseEntity gives ordinary entities a Guid key and implements both interfaces so I do not need to repeat the boilerplate on every table.

```
public abstract class BaseEntity : IAuditable, ISoftDelete
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public DateTimeOffset CreatedOn { get; set; } = DateTimeOffset.UtcNow;
    public string? CreatedBy { get; set; }
    public DateTimeOffset? ModifiedOn { get; set; }
    public string? ModifiedBy { get; set; }
    public bool IsDeleted { get; set; }
    public DateTimeOffset? DeletedOn { get; set; }
    public string? DeletedBy { get; set; }
}
```

4.2 Identity entities

ApplicationUser extends IdentityUser<Guid> and additionally implements IAuditable and ISoftDelete so the same audit columns flow through the Users table. I added the profile fields the UI asks for (FirstName, LastName, MiddleName, TitleId, GenderId, DateOfBirth, Address, ProfilePhotoUrl), an IsActive flag, and a small pair of deactivation fields. I also exposed a computed FullName property that the UI uses everywhere.

```
public class ApplicationUser : IdentityUser<Guid>, IAuditable, ISoftDelete
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName  { get; set; } = string.Empty;
    public string? MiddleName { get; set; }

    public Guid?   TitleId { get; set; }
```

```
    public Title?  Title   { get; set; }
    public Guid?   GenderId { get; set; }
    public Gender? Gender   { get; set; }

    public DateTime? DateOfBirth { get; set; }
    public string?   Address     { get; set; }
    public string?   ProfilePhotoUrl { get; set; }

    public bool IsActive { get; set; } = true;
    public DateTimeOffset? DeactivatedOn { get; set; }
    public string?         DeactivationReason { get; set; }

    /* IAuditable + ISoftDelete members, omitted for brevity */

    public string FullName => string.IsNullOrWhiteSpace(MiddleName)
        ? $"{FirstName} {LastName}".Trim()
        : $"{FirstName} {MiddleName} {LastName}".Trim();
}
```

ApplicationRole and ApplicationUserRole are the same pattern: extend the Identity base types, add description / audit fields, and (for the join table) track AssignedOn and AssignedBy so there is a breadcrumb for who added a user to a role and when.

4.3 Lookup tables — no enums

Anything that would normally be a C# enum I modelled as a database table. This makes the data translatable, extensible by data rather than by code, and visible in reports. For Sprint 1 those tables are Title and Gender:

```
public class Gender : BaseEntity
{
    public string Name        { get; set; } = string.Empty;
    public string Code        { get; set; } = string.Empty;
    public int    DisplayOrder { get; set; }
    public ICollection<ApplicationUser> Users { get; set; } = [];
}
```

Title follows the same shape minus the Code column.

4.4 Role names — constants, not an enum

Role membership is validated against strings everywhere in ASP.NET (the Authorize attribute and policies expect string role names), but I did not want magic strings scattered through the code. So Roles is a static class whose members are const strings:

```
public static class Roles
{
    public const string SuperAdmin         = nameof(SuperAdmin);
    public const string HeadTeacher        = nameof(HeadTeacher);
    public const string Teacher            = nameof(Teacher);
    public const string SchoolBursar       = nameof(SchoolBursar);
    public const string SchoolStoreKeeper  = nameof(SchoolStoreKeeper);
```

```
    public const string Parent             = nameof(Parent);
    public const string Student            = nameof(Student);

    public static readonly IReadOnlyList<string> All =
        [SuperAdmin, HeadTeacher, Teacher, SchoolBursar,
         SchoolStoreKeeper, Parent, Student];
}
```

The seeder iterates Roles.All to create exactly these rows on every startup, and the Razor pages spell role names as Roles.SuperAdmin so the compiler catches typos.

5. The Application layer

Application holds contracts — service interfaces and DTOs — with no EF Core, no ASP.NET, and no Razor. Its sole dependency is Domain.

5.1 Cross-cutting abstractions

Two helpers are used by almost every feature:

```
public interface ICurrentUser
{
    Guid?   UserId { get; }
    string? UserName { get; }
    bool    IsAuthenticated { get; }
}
```

ICurrentUser is implemented in the Web project (where HttpContext is available) but consumed by ApplicationDbContext in Infrastructure so that audit columns can be stamped with the signed-in user's name without leaking HttpContext into EF Core.

```
public class OperationResult
{
    public bool Succeeded { get; init; }
    public IReadOnlyList<string> Errors { get; init; } = [];

    public static OperationResult Success() => new() { Succeeded = true };
    public static OperationResult Failure(params string[] errors) =>
        new() { Succeeded = false, Errors = errors };
}

public class OperationResult<T> : OperationResult
{
    public T? Data { get; init; }
    public static OperationResult<T> Success(T data) =>
        new() { Succeeded = true, Data = data };
}
```

OperationResult lets services return a predictable shape — success + data or failure + an array of messages — without throwing exceptions for expected validation failures. The Blazor pages turn those messages into RadzenNotification pop-ups.

5.2 Data transfer objects

Each feature has small request / response DTOs in Application/Users/Dtos/. DataAnnotations attributes live on the requests so that Blazor's DataAnnotationsValidator can use them without re-declaring the rules on the page.

- UserDto — flat projection returned to the UI, including TitleName, GenderName, FullName, IsActive, and the role names.
- CreateUserRequest — Username, Email, Password, ConfirmPassword, profile fields, IsActive, and Roles (a List<string>). Roles carries [Required] and [MinLength(1)] so the form cannot be submitted without a role.
- UpdateUserRequest — profile + contact fields, but no password and no roles (role changes go through the dedicated assign-roles page).
- UserListFilter — paging inputs: SearchTerm, Role, IsActive, Skip, Take, OrderBy. Plus a generic PagedResult<T>.
- LookupDto / RoleDto — simple Id/Name (+ Code, Description) shapes for dropdowns and tables.

5.3 Service contracts

Two interfaces own the entire sprint's behaviour:

```
public interface IUserService
{
    Task<PagedResult<UserDto>> ListAsync(UserListFilter filter, CancellationToken ct =
default);
    Task<UserDto?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult<Guid>> CreateAsync(CreateUserRequest request, CancellationToken ct
= default);
    Task<OperationResult>       UpdateAsync(UpdateUserRequest request, CancellationToken ct
= default);
    Task<OperationResult>       SetActiveAsync(Guid userId, bool isActive, string? reason,
CancellationToken ct = default);
    Task<OperationResult>       SoftDeleteAsync(Guid userId, CancellationToken ct =
default);
    Task<OperationResult>       AssignRolesAsync(Guid userId, IEnumerable<string> roles,
CancellationToken ct = default);
    Task<OperationResult>       SetPasswordAsync(Guid userId, string newPassword,
CancellationToken ct = default);
    Task<IReadOnlyList<string>> GetRolesAsync(Guid userId, CancellationToken ct = default);
}
```

```
public interface ILookupService
{
    Task<IReadOnlyList<LookupDto>> GetGendersAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetTitlesAsync(CancellationToken ct = default);
    Task<IReadOnlyList<RoleDto>>   GetRolesAsync(CancellationToken ct = default);
}
```

Notice that every method takes a CancellationToken with a default value. Blazor Server components pass the component's cancellation token naturally, so long-running queries stop when the user navigates away.

6. The Infrastructure layer

Infrastructure does the heavy lifting: it owns the EF Core DbContext, all persistence, and the real implementations of the service interfaces. It depends on Application and therefore transitively on Domain.

6.1 ApplicationDbContext

I started with IdentityDbContext because Identity needs its own DbContext with the UserStore and RoleStore relationships pre-configured. I used the eight-generic-argument overload so I could supply my own ApplicationUserRole join entity (which carries AssignedOn / AssignedBy):

```
public class ApplicationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    ICurrentUser currentUser)
    : IdentityDbContext<
        ApplicationUser,
        ApplicationRole,
        Guid,
        IdentityUserClaim<Guid>,
        ApplicationUserRole,
        IdentityUserLogin<Guid>,
        IdentityRoleClaim<Guid>,
        IdentityUserToken<Guid>>(options)
{
    public DbSet<Gender> Genders => Set<Gender>();
    public DbSet<Title>  Titles  => Set<Title>();
```

The constructor takes ICurrentUser alongside the options so that the audit logic can reach the signed-in user without static state. I pass it into a primary constructor (a C# 12 feature) to keep the plumbing tight.

6.2 Model configuration

In OnModelCreating I do three things: rename the Identity tables to friendlier names, apply column constraints, and attach soft-delete query filters. A small extract:

```
builder.Entity<ApplicationUser>(b =>
{
    b.ToTable("Users");
    b.Property(u => u.FirstName).HasMaxLength(80).IsRequired();
    b.Property(u => u.LastName).HasMaxLength(80).IsRequired();
    b.Ignore(u => u.FullName);

    b.HasOne(u => u.Title).WithMany(t => t.Users)
        .HasForeignKey(u => u.TitleId)
        .OnDelete(DeleteBehavior.SetNull);
```

```
    b.HasOne(u => u.Gender).WithMany(g => g.Users)
        .HasForeignKey(u => u.GenderId)
        .OnDelete(DeleteBehavior.SetNull);

    b.HasIndex(u => u.IsDeleted);
    b.HasQueryFilter(u => !u.IsDeleted);
});
```

The HasQueryFilter call is the critical line — every LINQ query against Users automatically filters out soft-deleted rows. Admin reports that need to see deleted rows can call IgnoreQueryFilters() explicitly.

For the lookup tables I wrote a tiny ConfigureLookup<T> helper that applies the same pattern (table name, audit column lengths, soft-delete filter) so I do not repeat myself across Gender and Title.

6.3 Auditing and soft delete inside SaveChanges

Rather than stamp CreatedBy, ModifiedBy, DeletedBy manually in every service, I override SaveChanges / SaveChangesAsync to do it in one place:

```
private void ApplyAuditAndSoftDelete()
{
    var now      = DateTimeOffset.UtcNow;
    var userName = currentUser.UserName ?? "system";

    foreach (var entry in ChangeTracker.Entries())
    {
        if (entry.Entity is IAuditable a)
        {
            if (entry.State == EntityState.Added)
            {
                a.CreatedOn = a.CreatedOn == default ? now : a.CreatedOn;
                a.CreatedBy ??= userName;
            }
            else if (entry.State == EntityState.Modified)
            {
                a.ModifiedOn = now;
                a.ModifiedBy = userName;
            }
        }

        if (entry.Entity is ISoftDelete s && entry.State == EntityState.Deleted)
        {
            entry.State = EntityState.Modified;  // turn Delete into Update
            s.IsDeleted = true;
            s.DeletedOn = now;
            s.DeletedBy = userName;
        }
    }
}
```

Rewriting Deleted → Modified is the trick that turns every db.Remove() call into a soft delete without services needing to know about it.

6.4 Dependency injection — AddInfrastructure

I expose a single extension method that the Web project calls from Program.cs. It registers the DbContext, configures Identity with the password and lockout rules, and wires up the service implementations.

```
public static IServiceCollection AddInfrastructure(
    this IServiceCollection services, IConfiguration configuration)
{
    var connectionString = configuration.GetConnectionString("DefaultConnection")
        ?? throw new InvalidOperationException("Connection string 'DefaultConnection' not
found.");

    services.AddDbContext<ApplicationDbContext>(options =>
        options.UseSqlServer(connectionString, sql =>
            sql.MigrationsAssembly(typeof(ApplicationDbContext).Assembly.FullName)));

    services
        .AddIdentityCore<ApplicationUser>(o =>
        {
            o.SignIn.RequireConfirmedAccount = false;
            o.User.RequireUniqueEmail        = true;
            o.Password.RequireDigit          = true;
            o.Password.RequireLowercase      = true;
            o.Password.RequireUppercase      = true;
            o.Password.RequireNonAlphanumeric = true;
            o.Password.RequiredLength         = 8;
            o.Lockout.AllowedForNewUsers      = true;
            o.Lockout.MaxFailedAccessAttempts = 5;
            o.Lockout.DefaultLockoutTimeSpan  = TimeSpan.FromMinutes(15);
        })
        .AddRoles<ApplicationRole>()
        .AddEntityFrameworkStores<ApplicationDbContext>()
        .AddSignInManager()
        .AddDefaultTokenProviders();

    services.AddScoped<IUserService,   UserService>();
    services.AddScoped<ILookupService, LookupService>();

    return services;
}
```

6.5 UserService

UserService uses the Identity UserManager / RoleManager for the operations they know about (create, password, role membership) and drops down to ApplicationDbContext directly for list queries that need flexible filtering. Notable decisions:

- ListAsync joins UserRoles -> Roles manually rather than calling userManager.GetRolesAsync per user, because the latter would be N+1 queries for a paged list.
- CreateAsync validates uniqueness of UserName and Email up front, checks every requested role actually exists, then creates the user and assigns roles in two calls.

- SetActiveAsync flips IsActive, sets LockoutEnd to DateTimeOffset.MaxValue when deactivating (so the cookie is rejected immediately) and calls UpdateSecurityStampAsync so the revalidating provider will log the user out within its 30-minute window.
- SoftDeleteAsync does the same plus sets IsDeleted = true; the global query filter on Users means the row vanishes from ordinary queries.
- AssignRolesAsync diffs the current role set against the requested set and only adds / removes the difference, so the audit log stays clean.
- SetPasswordAsync uses a generated reset token rather than passing Identity's internal password-hash APIs — more conservative.

6.6 LookupService

LookupService is a thin projection layer — Genders ordered by DisplayOrder, Titles the same, and Roles ordered by Name. Each method returns LookupDto / RoleDto so the UI never sees the raw entities.

6.7 DatabaseInitializer — migrate and seed

A single static class drives first-run bootstrapping. Called from Program.cs after the DI container is built, it:

6. Applies any outstanding EF Core migrations (db.Database.MigrateAsync).
7. Seeds Genders (Male, Female) and nine Titles (Mr., Mrs., Miss, Ms., Dr., Prof., Chief, Alhaji, Alhaja) if those tables are empty.
8. Iterates Roles.All and creates any missing ApplicationRole rows with descriptions and IsSystemRole = true.
9. Creates the default SuperAdmin user (superadmin@naijaprimeschool.ng / Admin@12345) if none exists, and adds them to the SuperAdmin role.

This is what makes the app runnable on a fresh database without any manual SQL. The README warns administrators to reset that initial password on first sign-in.

6.8 Creating the first migration

I installed dotnet-ef globally (once per machine) and then added the initial migration. The migration lives in the Infrastructure project but is driven by the Web project because that is where the connection string lives:

```
dotnet tool install --global dotnet-ef --version 10.0.0

dotnet ef migrations add InitialCreate \
    --project src/NaijaPrimeSchool.Infrastructure \
    --startup-project src/NaijaPrimeSchool.Web \
    --output-dir Persistence/Migrations
```

Because DatabaseInitializer.InitializeAsync runs on every startup and calls MigrateAsync, I did not need a separate 'dotnet ef database update' step — but it is still available if I want to apply migrations out-of-band.

7. The Web host — wiring Blazor + Identity

The Web project is the composition root. It boots Blazor, registers Radzen, turns on authentication, builds the authorization policies, and finally runs the database initializer.

7.1 Program.cs walk-through

Reading Program.cs top-to-bottom tells you almost everything:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents()
    .AddInteractiveWebAssemblyComponents()
    .AddAuthenticationStateSerialization();

builder.Services.AddCascadingAuthenticationState();
builder.Services.AddScoped<IdentityRedirectManager>();
builder.Services.AddScoped<AuthenticationStateProvider,
    IdentityRevalidatingAuthenticationStateProvider>();

builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUser, CurrentUserAccessor>();

builder.Services.AddAuthentication(o =>
{
    o.DefaultScheme       = IdentityConstants.ApplicationScheme;
    o.DefaultSignInScheme = IdentityConstants.ExternalScheme;
}).AddIdentityCookies();

builder.Services.ConfigureApplicationCookie(o =>
{
    o.LoginPath         = "/Account/Login";
    o.AccessDeniedPath  = "/Account/AccessDenied";
    o.LogoutPath        = "/Account/Logout";
    o.SlidingExpiration = true;
    o.ExpireTimeSpan    = TimeSpan.FromHours(8);
});

builder.Services.AddInfrastructure(builder.Configuration);

builder.Services.AddAuthorization(o =>
{
    o.AddPolicy("ManageUsers", p => p.RequireRole(Roles.SuperAdmin));
});

builder.Services.AddRadzenComponents();
```

After app = builder.Build() the request pipeline is:

```
if (app.Environment.IsDevelopment())
    app.UseWebAssemblyDebugging();
else
{
```

```
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}

app.UseStatusCodePagesWithReExecute("/not-found", createScopeForStatusCodePages: true);
app.UseHttpsRedirection();
app.UseAntiforgery();

app.MapStaticAssets();
app.MapRazorComponents<App>()
   .AddInteractiveServerRenderMode()
   .AddInteractiveWebAssemblyRenderMode()
   .AddAdditionalAssemblies(typeof(NaijaPrimeSchool.Web.Client._Imports).Assembly);

app.MapAdditionalIdentityEndpoints();

using (var scope = app.Services.CreateScope())
{
    await DatabaseInitializer.InitializeAsync(scope.ServiceProvider);
}

app.Run();
```

The single authorization policy, ManageUsers, is what guards every user management page with [Authorize(Roles = Roles.SuperAdmin)] — I also kept the direct role form on the pages as a defensive belt-and-braces.

7.2 CurrentUserAccessor — bridging HttpContext to DI

ICurrentUser is implemented in the Web project because only here is HttpContext available:

```
public sealed class CurrentUserAccessor(IHttpContextAccessor ctx) : ICurrentUser
{
    public Guid? UserId
    {
        get
        {
            var raw = ctx.HttpContext?.User.FindFirstValue(ClaimTypes.NameIdentifier);
            return Guid.TryParse(raw, out var id) ? id : null;
        }
    }
    public string? UserName => ctx.HttpContext?.User.Identity?.Name;
    public bool    IsAuthenticated => ctx.HttpContext?.User.Identity?.IsAuthenticated ==
true;
}
```

7.3 Logout endpoint

I mapped a POST /Account/Logout endpoint so the header logout button is a traditional form submission (anti-forgery token included). It calls SignInManager.SignOutAsync and redirects back to login:

```
accountGroup.MapPost("/Logout", async (
```

```
    [FromServices] SignInManager<ApplicationUser> signInManager,
    [FromForm] string? returnUrl) =>
{
    await signInManager.SignOutAsync();
    return TypedResults.LocalRedirect($"~/{returnUrl ?? "Account/Login"}");
});
```

7.4 Revalidating authentication state

A plain cookie would let a deactivated user keep clicking around until the cookie expired. I do not want that. IdentityRevalidatingAuthenticationStateProvider extends the framework's RevalidatingServerAuthenticationStateProvider and re-checks the user against the database every 30 minutes — if the user is deleted, deactivated, or their security stamp has changed, the provider returns false and Blazor forces a sign-out:

```
protected override TimeSpan RevalidationInterval => TimeSpan.FromMinutes(30);

protected override async Task<bool> ValidateAuthenticationStateAsync(
    AuthenticationState authenticationState, CancellationToken ct)
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var userManager =
scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();
    var user = await userManager.GetUserAsync(authenticationState.User);
    if (user is null || user.IsDeleted || !user.IsActive) return false;
    /* check security stamp matches current stamp ... */
    return principalStamp == userStamp;
}
```

8. The Razor UI

Every page is an InteractiveServer Blazor component. I kept things server-rendered because Sprint 1 is administrative and small, and InteractiveServer avoids the 'load WASM runtime' latency. The Web.Client project is present and wired so future features can opt into Auto interactivity per page.

8.1 App and Routes

App.razor is the html shell. It pre-loads Radzen's material-base stylesheet, my app.css, and the Blazor framework script.

```
<link rel="stylesheet" href="_content/Radzen.Blazor/css/material-base.css" />
<link rel="stylesheet" href="@Assets["app.css"]" />
<link rel="stylesheet" href="@Assets["NaijaPrimeSchool.Web.styles.css"]" />
...
<Routes />
<script src="_content/Radzen.Blazor/Radzen.Blazor.js"></script>
<script src="@Assets["_framework/blazor.web.js"]"></script>
```

Routes.razor uses AuthorizeRouteView so an unauthenticated user hitting a protected page gets sent to the login flow (via RedirectToLogin — a tiny client component that calls NavigationManager.NavigateTo with forceLoad: true).

8.2 _Imports.razor

Every page shares one set of usings, so most components do not need any @using directives of their own:

```
@using Microsoft.AspNetCore.Authorization
@using Microsoft.AspNetCore.Components.Authorization
@using Microsoft.AspNetCore.Components.Forms
@using Radzen
@using Radzen.Blazor
@using NaijaPrimeSchool.Domain.Identity
@using NaijaPrimeSchool.Application.Common
@using NaijaPrimeSchool.Application.Users
@using NaijaPrimeSchool.Application.Users.Dtos
@using NaijaPrimeSchool.Web.Components.Account
@using NaijaPrimeSchool.Web.Components.Layout
```

8.3 MainLayout and NavMenu

MainLayout wraps authenticated pages with a RadzenLayout: header on top (brand, sign-out form), sidebar on the left (NavMenu), and a RadzenBody for page content. The whole tree is inside <AuthorizeView> so the sidebar is hidden on login / access-denied screens.

NavMenu renders different items depending on which roles the signed-in user has. Only SuperAdmins see the User Management group; other role groups are stubbed as Disabled='true' items, reserving the space in the sidebar for future sprints.

8.4 Sign-in page — Login.razor

The login page is a classic server-rendered EditForm with FormName="login" so Blazor binds the POST back to the same component. It accepts either a username or an email address, checks that the user is not soft-deleted or deactivated, and calls SignInManager.PasswordSignInAsync with lockoutOnFailure: true so the Identity lockout policy applies.

```
var user = await UserManager.FindByNameAsync(Input.Email)
       ?? await UserManager.FindByEmailAsync(Input.Email);

if (user is null || user.IsDeleted) { errorMessage = "Invalid sign in attempt."; return; }
if (!user.IsActive) { errorMessage = "Your account has been deactivated..."; return; }

var result = await SignInManager.PasswordSignInAsync(
    user.UserName!, Input.Password, Input.RememberMe, lockoutOnFailure: true);
```

On success the page redirects to the ReturnUrl query-string parameter or to the dashboard. Lockouts surface a friendly message.

8.5 The Users list — Users.razor

The list is a RadzenDataGrid with server-side paging. Above the grid I placed a filter bar — a text box bound to SearchTerm, a role dropdown, and an Active/Inactive dropdown. Every change calls LoadAsync, which rebuilds a UserListFilter and hands it to IUserService.ListAsync.

- Rows show FullName with the email underneath it (via the Template slot), followed by username, a horizontal stack of role badges, and a green / grey status badge.
- Row actions live in the right-most column: Edit, Assign roles, and either Deactivate or Activate depending on current status.
- Deactivate asks for confirmation via DialogService.Confirm before calling IUserService.SetActiveAsync — no accidental clicks.
- After every action I reload the page via LoadAsync and push a Radzen notification so the admin gets feedback.

8.6 Add user — CreateUser.razor

The add-user form is a RadzenTemplateForm bound to a CreateUserRequest. It uses <DataAnnotationsValidator /> so all the [Required], [EmailAddress], [StringLength] and [Compare] attributes I put on the DTO are enforced with no extra ceremony. The role selector is a RadzenCheckBoxList bound directly to model.Roles (via Value / ValueChanged / ValueExpression) — binding to model.Roles, not a separate field, is important so that DataAnnotationsValidator sees the selection and the [MinLength(1)] validation on Roles actually prevents submit.

On submit the page calls IUserService.CreateAsync, shows a Radzen notification on success or spreads the OperationResult.Errors into a red notification on failure, then navigates back to /users.

8.7 Edit user — EditUser.razor

The edit page routes on /users/{Id:guid}, loads the user via IUserService.GetByIdAsync, and presents the same profile fields as the add form (minus username, password, and roles, which are handled elsewhere). The header carries Activate / Deactivate buttons and a shortcut to the role-assignment page.

8.8 Assign roles — UserRoles.razor

This page is intentionally small: a RadzenCheckBoxList of every role bound to the user's current role set, plus Save / Cancel. On save it calls IUserService.AssignRolesAsync, which diffs and applies only the change set so audit history stays clean.

8.9 Roles catalogue — RolesPage.razor

A read-only grid at /roles lists every role with its description and a System / Custom badge — useful for admins who want to see the full taxonomy at a glance. All rows are System because Sprint 1 seeds them from code.

8.10 Reset password — ResetPasswordDialog.razor

Admin-initiated password reset is a Radzen dialog hosted inside EditUser, not a separate route. It asks for a new password (twice), validates the confirmation, and calls IUserService.SetPasswordAsync, which

internally generates an Identity reset token and applies it — far safer than touching the password hash directly.

8.11 Home dashboard

The home page shows the signed-in user's name and four stat tiles (total users, roles configured, active users, deactivated users). It also renders SuperAdmin-only quick-action buttons for Manage users and Add user. All stats are computed once on OnInitializedAsync with the existing services — no new endpoint was needed.

9. Theming — the Nigerian green + gold skin

Radzen ships with neutral base stylesheets. I layered a thin custom skin on top (wwwroot/app.css) using CSS custom properties so every page can reference --nps-green / --nps-gold / --nps-ink-500 and stay consistent. Only three concerns matter:

- Colour tokens — deep Nigerian green for primary, gold for accents, a muted ink palette for text and borders.
- Layout primitives — .nps-page-container, .nps-page-header, .nps-card, .nps-form-grid (two-column responsive grid), .nps-form-actions. Used on every page for a consistent rhythm.
- The auth page — .nps-auth-page and .nps-auth-card give the login screen a centred, shadowed card with a brand mark.

I deliberately avoided tailwind / SASS for Sprint 1 because the Radzen + CSS-variables approach is enough for a small admin area, and it means there is no additional build step for the front-end.

10. Running the application locally

10.1 appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=NaijaPrimeSchool;Trusted_Connection=True;MultipleActiveResultSets=true;T
rustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" }
  },
  "AllowedHosts": "*"
}
```

For SQL Server LocalDB change Server=. to Server=(localdb)\MSSQLLocalDB. The connection string never contains real credentials in the committed file — production secrets live in user-secrets or environment variables.

10.2 Restore, build, run

```
dotnet restore
dotnet build
dotnet run --project src/NaijaPrimeSchool.Web
```

On first run the DatabaseInitializer creates the schema, seeds the lookups, roles, and the default SuperAdmin. The console prints the URL to open (typically https://localhost:7xxx). I signed in with:

```
Email:    superadmin@naijaprimeschool.ng
Password: Admin@12345   ← change immediately in Users → Edit → Reset password
```

10.3 A suggested VS Code launch configuration

C# Dev Kit auto-generates .vscode/launch.json the first time I press F5, but I keep a hand-written version so everyone on the team gets the same experience:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": ".NET Core Launch (web)",
      "type": "coreclr",
      "request": "launch",
      "preLaunchTask": "build",
      "program":
"${workspaceFolder}/src/NaijaPrimeSchool.Web/bin/Debug/net10.0/NaijaPrimeSchool.Web.dll",
      "cwd":      "${workspaceFolder}/src/NaijaPrimeSchool.Web",
      "stopAtEntry": false,
      "serverReadyAction": { "action": "openExternally", "pattern": "\\bNow listening on:\
\s+(https?://\\S+)" },
      "env": { "ASPNETCORE_ENVIRONMENT": "Development" }
    }
  ]
}
```

11. Source control and pushing to GitHub

The final step is putting the sprint under version control. I did everything from the VS Code terminal.

11.1 Writing a sensible .gitignore

I hand-rolled a .gitignore covering the usual .NET build outputs (bin/, obj/), Visual Studio and Rider user state (.vs/, .vscode/, .idea/, *.suo, *.user), NuGet caches, publish profiles, appsettings.*.Local.json and appsettings.Production.json, plus secrets like .env, *.pfx, and secrets.json. Nothing sensitive leaves the repo.

11.2 First commit on the sprint branch

```
git init -b main
git remote add origin https://github.com/benjaminsqlserver/NaijaPrimeSchool.git
```

```
git checkout -b sprint/1-identity-and-user-management

git add .gitignore README.md NaijaPrimeSchool.slnx src
git status --short              # sanity check — no bin/, obj/, or .vs/

git commit -m "Sprint 1: identity and user management foundation"
```

I avoided git add -A so nothing unexpected would slip in. The status output confirmed only source, the solution file, the README, and the .gitignore were staged.

11.3 Pushing to the empty GitHub repo

I had already created an empty public repository at github.com/benjaminsqlserver/NaijaPrimeSchool in the GitHub UI (without a README or .gitignore — the local commit is the initial state). Then:

```
git push -u origin sprint/1-identity-and-user-management

# Populate main with the same commit so GitHub's default branch is not empty
git branch main sprint/1-identity-and-user-management
git push -u origin main
```

Git Credential Manager prompted me once for GitHub credentials via a browser pop-up; subsequent pushes re-use the stored token. Sprint 1 is now live on both main and sprint/1-identity-and-user-management.

12. Appendix — project map

For reference, the key files produced during Sprint 1 and what each is for:

Path	Purpose
src/NaijaPrimeSchool.Domain/Common/BaseEntity.cs	Base id + audit + soft-delete fields.
src/NaijaPrimeSchool.Domain/Identity/ApplicationUser.cs	IdentityUser<Guid> with profile, audit, and soft-delete.
src/NaijaPrimeSchool.Domain/Identity/Roles.cs	String constants for role names — not an enum.
src/NaijaPrimeSchool.Application/Common/ICurrentUser.cs	Abstraction over the signed-in user, implemented in Web.
src/NaijaPrimeSchool.Application/Common/OperationResult.cs	Success/failure shape returned by services.
src/NaijaPrimeSchool.Application/Users/IUserService.cs	User management contract (CRUD + roles + password).
src/NaijaPrimeSchool.Infrastructure/Persistence/ApplicationDbContext.cs	IdentityDbContext + query filters + SaveChanges auditing.
src/NaijaPrimeSchool.Infrastructure/Persistence/DatabaseInitializer.cs	Migrate + seed lookups, roles, SuperAdmin.
src/NaijaPrimeSchool.Infrastructure/Services/UserService.cs	Implementation using UserManager / RoleManager / DbContext.
src/NaijaPrimeSchool.Infrastructure/	AddInfrastructure — DI composition for the app.

DependencyInjection.cs	
src/NaijaPrimeSchool.Web/Program.cs	Host wiring: Identity, cookies, Radzen, policies, endpoints, DB init.
src/NaijaPrimeSchool.Web/Services/CurrentUserAccessor.cs	HttpContext-backed ICurrentUser implementation.
src/NaijaPrimeSchool.Web/Components/Account/Pages/Login.razor	Sign-in page (EditForm + SignInManager).
src/NaijaPrimeSchool.Web/Components/Pages/Users/	List / Create / Edit / Roles / Reset password.
src/NaijaPrimeSchool.Web/Components/Layout/MainLayout.razor	Radzen shell with sidebar + header.
src/NaijaPrimeSchool.Web/wwwroot/app.css	Green + gold school skin.

— End of Sprint 1 implementation guide. Sprint 2 picks up the academic domain (sessions, terms, classes, subjects).